Universidade Federal de Uberlândia – UFU
Faculdade de Computação – FACOM
Prof. Dr. Rivalino

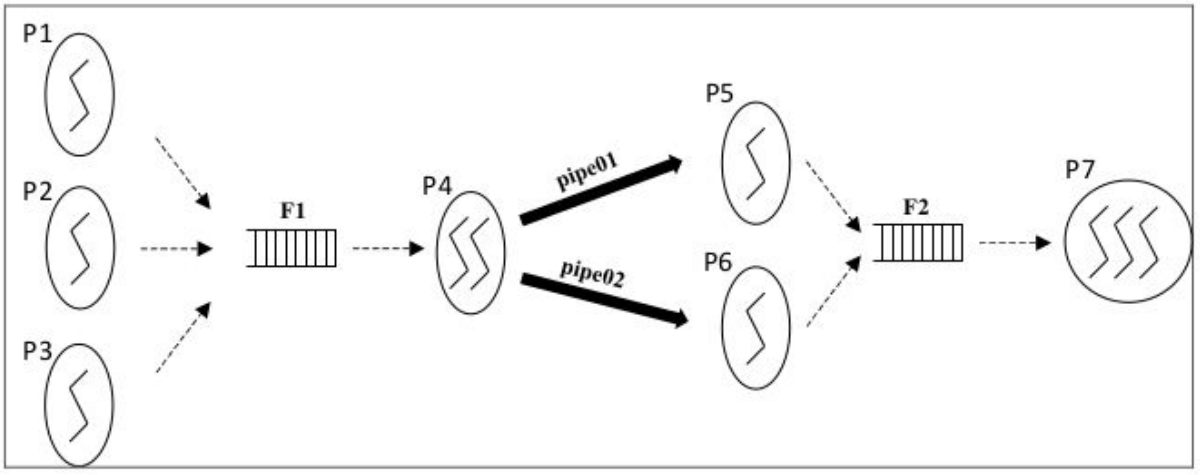


Questionário Unidade 3

Hericles Felipe Ferraz - 11811EMT022

Uberlândia
29/11/2020

Questão: Implementar um programa que realize o processamento ilustrado na figura abaixo



Os processos P1, P2 e P3 são singlethreads. Cada um desses processos se comunica com o processo P4 por meio de uma área de memória compartilhada (shared memory). Essa área compartilhada deve ser usada como uma fila (F1) do tipo FIFO (first-in-first-out), a qual armazena valores do tipo inteiro. A capacidade da fila é de 10 valores.

Os processos P1, P2 e P3 são produtores e o processo P4 é consumidor nessa comunicação. P4 sempre aguarda um sinal (SIGUSR1) para consumir dados da F1. Esse sinal é enviado para P4 quando F1 estiver cheia, ou seja, com 10 valores. O sinal deve ser enviado para P4 pelo processo que inseriu o décimo valor em F1. Note que neste caso F1 somente receberá valores após P4 retirar o último (décimo) valor de F1, deixando-a vazia para receber novos valores.

Os valores inteiros inseridos em F1 devem ser gerados de forma aleatória na faixa de 1 até 1000.

O acesso a F1 deve ocorrer de forma exclusiva utilizando o mecanismo de semáforo entre os processos envolvidos.

O processo P4 possui duas threads (t1 e t2), onde ambas as threads retiram valores de F1 e enviam para P5 (t1) e P6 (t2), respectivamente, utilizando o mecanismo de pipe. Os processos P5 e P6 ao receberem valores de P4 os enviam para P7 utilizando da fila F2, também implementada como shared memory. Neste caso, o controle ao acesso de F2 deve ser implementado usando um mecanismo de exclusão mútua baseado em espera ocupada (busy wait).

Diferente de F1, a fila F2 deve ser consumida na medida em que entram novos valores. Se a fila estiver cheia (máximo 10 valores), então os processos P5 e P6 devem aguardar a retirada de ao menos um elemento da fila, por P7, para inserirem novos valores.

Todas as três threads de P7 retiram valores de F2 e os imprimem na tela. Após P7 imprimir 10000 valores, o programa deve imprimir o seguinte relatório:

a) Tempo total de execução do programa.

b) Quantidade de valores processados por P5 e P6.

c) Dentre os valores impressos, informar o valor que mais se repetiu (moda), o valor mínimo e o valor máximo.

Resolução:

Apesar do código skeleton.c ter sido disponibilizado, optei por fazer a atividade em códigos distintos. O motivo desta escolha era para tentar me aproximar o máximo possível do funcionamento dos tópicos e serviços oferecidos no ROS.

A seguinte estrutura foi reproduzida:

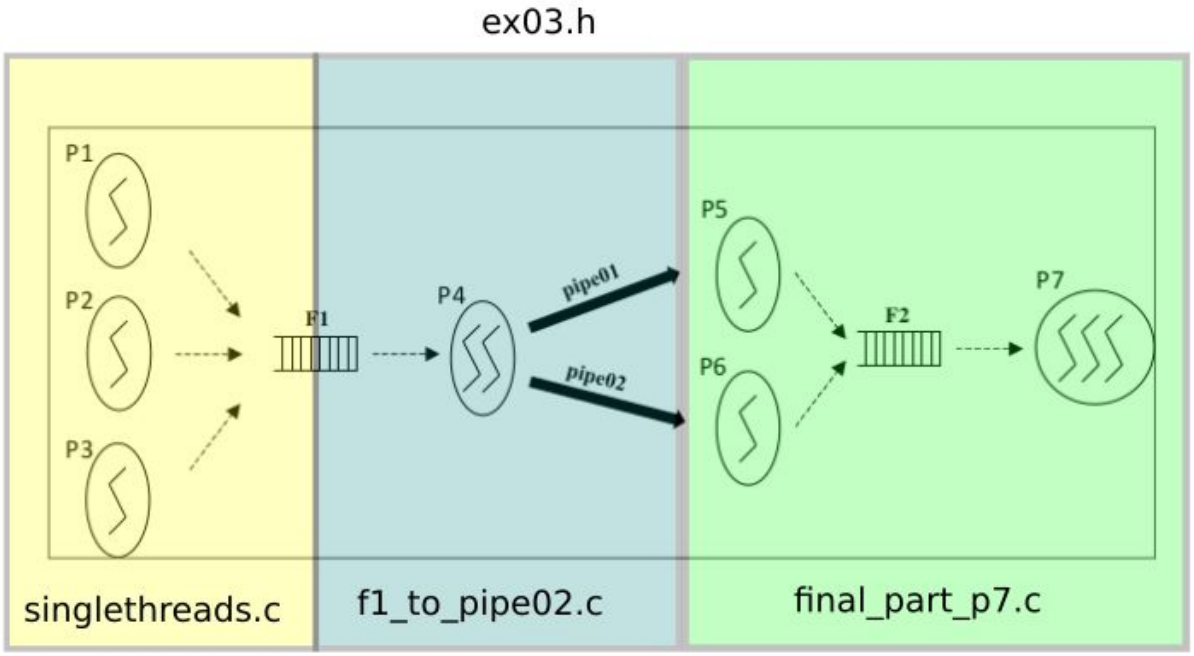


Fig 1 - Estrutura definida para a atividade

Onde conforme mostrado na Fig 1, cada cor representa um código, e o arquivo de cabeçalho ex03.h tem definições de variáveis ou/e funções para todos os códigos.c.

O código singlethreads.c cria os processos p1, p2 e p3. Após a criação destes processos é passado os valores inteiros para a fila f1 por meio da memória compartilhada.

O código f1_to_pipe02.c conecta o processo 4 para compartilhar os recursos de p1, p2 e p3* e os processos p5 e p6 por meio dos canais pipe01 e pipe02.

Já o código final_part_p7.c cria os processos p5, p6 e p7. O processo 4 somente é acessado quando recebe um signal. Os recursos são compartilhados pela fila f2.

Como executar: Os valores inteiros são passados principalmente pelo f1_to_pipe02, que é a principal ponte entre toda a cadeia de processos definida na Fig 1. Estes valores inteiros são gerados no singlethreads.c, e são finalizados no processo 7 no código final_part_p7.c.

Desta forma, a forma de compilar e executar é dada pela seguinte ordem:

$ gcc -o f1_to_pipe02 -pthread f1_to_pipe02.c

$ gcc -o final_part_p7 -pthread final_part_p7.c

$ gcc -o singlethreads -pthread singlethreads.c

A execução deve ser feita na mesma ordem da compilação apresentada anteriormente, sendo:

$ ./f1_to_pipe02

$ ./final_part_p7

$ ./singlethreads

Ao fim desta execução, tem-se a Fig 2 como resultado:

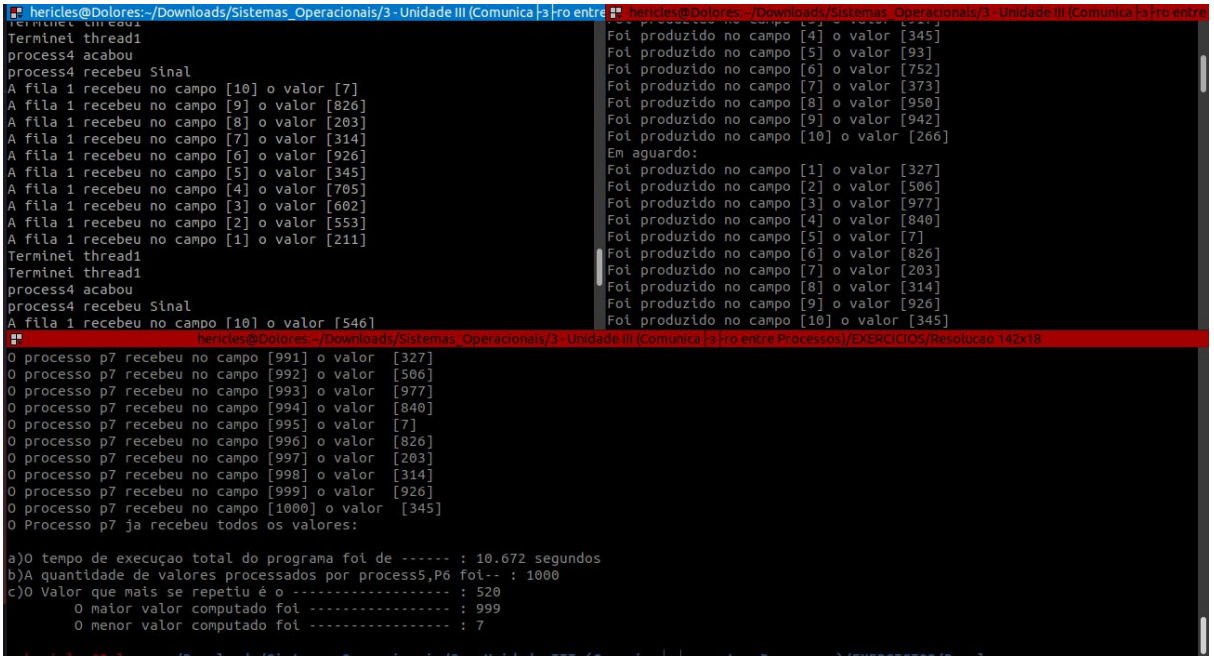


Fig 2 - Resultados Obtidos

Conforme mostrado na Fig 2, as respostas são:

a) O tempo de execuçao fpo de 10,672 segundos

b) A quantidade de valores processados por p5 e p6 foi de: 1000

c) Moda : 520

Valor Máximo: 999

Valor Mínimo: 7

Códigos Utilizados:

ex03.h

```
#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <sys/types.h>

#include <sys/shm.h>

#include <semaphore.h>

#include <signal.h>

#include <pthread.h>

#include <time.h>


#define MEM_F1 4096

#define MEM_F2 4096


//Criando area de memoria compartilhada para f1

struct shared *f1_shared_area_ptr;

struct shared{

   sem_t mutex;

   int num;

   int int_value[10];

   int PID;

};


//Criando area de memoria compartilhada para f1
```

```
struct shared_f2 *f2_shared_area_ptr;

struct shared_f2{

   int num;

   int int_value[10];

   int counter_p5_p6;

   sem_t mutex;

   clock_t t0;

};


//Responsavel pelo envio de t1 a p5 via PIPE

int pipe01[2];


//Responsavel pelo envio de t1 a p6 via PIPE

int pipe02[2];


//Funcoes e variaiveis para o codigo singlethreads.c

void singleProcess();

int n;

int nF1;


//Funcoes e variaiveis para o codigo f1_to_pipe02.c

void process4(int p);

void* thread1 ( void *ptr );

void* thread2 ( void *ptr );
```

```c
int max_values=1000, count = 0, size=0, counter_p5_p6=0;

int *moda = ((void*)0);

int *freq = ((void*)0);



//Variavel responsavel por contabilizar o tempo total

double total_time;



//Variaveis por tempo inicial e final

clock_t initial_time,final_time;



//funcao para interptetar os valores e definir a moda, valor maximo e
minimo

void dates_interpreter(int in);



void* three_threads_p7 (void *ptr);
```

f1_to_pipe02

```
f1_to_pipe02

/**

* Este codigo conecta f1 com process4 para compartilhar os recursos de p1, p2 e p3

* e os processos p5 e p6 por meio dos canais pipe01 e pipe02

**/



#include "ex03.h"



int main()

{

   //Preparando processos

   pid_t pid1;

   pid_t pid2;



     //chave de identificação para f1 para conectar singletheads.c e f1_to_pipe02.c

   key_t key=1234;

   void *shared_memory = (void *)0;

   int shmid;



   //Criação do segmento de memoria

   shmid = shmget(key,MEM_F1,0666|IPC_CREAT);

   //verificando situação

   if ( shmid == -1 )
```

```
{
    printf("shmget falhou\n");
    exit(-1);
}

//Recebendo endereço de memoria alocado
shared_memory = shmat(shmid,(void*)0,0);

//verificando situação
if (shared_memory == (void *) -1 )
{
    printf("shmat falhou\n");
    exit(-1);
}

f1_shared_area_ptr = (struct shared *) shared_memory;

if ( sem_init((sem_t*)&f1_shared_area_ptr->mutex,1,1) != 0 )
{
    printf("falha\n");
    exit(-1);
}

printf("A fila f1 está em espera: ");
```

```
   //Preparando troca de recursos
   sem_wait((sem_t*)&f1_shared_area_ptr->mutex);
   f1_shared_area_ptr->num = 10;
   f1_shared_area_ptr->PID=getpid();
   printf(" pid: %d\n",f1_shared_area_ptr->PID);
   sem_post((sem_t*)&f1_shared_area_ptr->mutex);


     //chave de identificação para f2 para conectar f1_to_pipe02.c e
final_part.c
   key_t keyF2=1243;
   void *shared_memory_f2 = (void *)0;
   int i;


   //Criação do segmento de memoria
   shmid = shmget(keyF2,MEM_F2,0666|IPC_CREAT);
   //verificando situação do shmid
   if ( shmid == -1 )
   {
       printf("shmget falhou\n");
       exit(-1);
   }


   //Recebendo endereço de memoria
   shared_memory_f2 = shmat(shmid,(void*)0,0);
```

```
//verificando situação shmat

if (shared_memory_f2 == (void *) -1 )

{

    printf("falha\n");

    exit(-1);

}



f2_shared_area_ptr = (struct shared_f2 *) shared_memory_f2;



if ( sem_init((sem_t*)&f2_shared_area_ptr->mutex,1,1) != 0 )

{

    printf("sem_init falhou\n");

    exit(-1);

}



sem_wait((sem_t*)&f2_shared_area_ptr->mutex);

f2_shared_area_ptr->num=0;

f2_shared_area_ptr->t0=clock();;

f2_shared_area_ptr->counter_p5_p6=0;



for(i=0;i<10;i++)

    f2_shared_area_ptr->int_value[i]=0;



sem_post((sem_t*)&f2_shared_area_ptr->mutex);
```

```
//Criando canal de comunicacao pipe01
if ( pipe(pipe01) == -1 ){ printf("Erro pipe()"); return -1; }
//Criando canal de comunicacao pipe02
if ( pipe(pipe02) == -1 ){ printf("Erro pipe()"); return -1; }


signal(SIGUSR1, process4);


//Criando processo filho
pid1=fork();
//Caso seja o processo filho "process5"
if ( pid1 == 0 )
{
    int process5, n5,sended_p5;
    while(1)
    {
        //Lendo f1
        read(pipe01[0],&process5,sizeof(int));
        sended_p5=0;
        while(sended_p5==0)
        {
            //Compartilhamento de recursos
            sem_wait((sem_t*)&f2_shared_area_ptr -> mutex);
            n5 = f2_shared_area_ptr->num;
            if(n5<10)
            {
```

```
                    f2_shared_area_ptr -> int_value[n5] = process5;

                    f2_shared_area_ptr ->num = (int)(n5+1);

                                                printf("process5   %d:   %d
\n",f2_shared_area_ptr->num,f2_shared_area_ptr->int_value[n5]);

                    n5=f2_shared_area_ptr->counter_p5_p6;

                    f2_shared_area_ptr->counter_p5_p6=n5+1;

                    sended_p5=1;

                }


            sem_post((sem_t*)&f2_shared_area_ptr->mutex);

            }

        }

    }

    else

    if (pid1 > 0 )

    {

        //Criando Processo Filho

        pid2=fork();

        if ( pid2 == 0 )

        {//SCRIPT FILHO 2 (P6)

            int P6,n,enviouP6;

            while(1)

            {

                read(pipe02[0],&P6,sizeof(int));

                enviouP6=0;
```

```
                while(enviouP6==0)
                {
                    sem_wait((sem_t*)&f2_shared_area_ptr->mutex);
                    n = f2_shared_area_ptr->num;
                    if(n<10)
                    {
                        f2_shared_area_ptr->int_value[n]=P6;
                        f2_shared_area_ptr->num = (int)(n+1);
                                                    printf("P6  %d:  %d
\n",f2_shared_area_ptr->num,f2_shared_area_ptr->int_value[n]);
                        n=f2_shared_area_ptr->counter_p5_p6;
                        f2_shared_area_ptr->counter_p5_p6=n+1;
                        enviouP6 = 1;
                    }
                    sem_post((sem_t*)&f2_shared_area_ptr->mutex);
                }
            }
        }
        else if (pid2 > 0 )
        {
            for(;;);
        }
    }
```

```
}


//funcao do process4 para acionar ao receber o sinal
void process4(int p)
{


   printf("process4 recebeu Sinal\n");
   pthread_t threads[2];
   int i;


   nF1=10;
   //Criando as threads
   pthread_create (&threads[0], NULL, thread1, NULL);
   pthread_create (&threads[1], NULL, thread2, NULL);


    for(i=0;i<2;i++)//ESPERANDO THREADS TERMINAREM
       pthread_join(threads[i], NULL);
       printf("process4 acabou\n");



   sem_wait((sem_t*)&f1_shared_area_ptr->mutex);
   //sleep(0.1);
   f1_shared_area_ptr->num=0;//NESTE PROGRAMA
   sem_post((sem_t*)&f1_shared_area_ptr->mutex);
}
```

```
void* thread1 ( void *ptr )

{



   int received,rsc,i,send;

   while(1)

   {

       send=0;

       sem_wait((sem_t*)&f1_shared_area_ptr->mutex);

       if ( nF1 > 0 )

       {

           send=1;

           received= f1_shared_area_ptr->int_value[0];



           for(i=0;i<nF1;i++)

           {

               rsc=f1_shared_area_ptr->int_value[i+1];

               f1_shared_area_ptr->int_value[i] = rsc;

           }

                   printf("A fila 1 recebeu no campo [%d] o valor
[%d]\n",nF1,received);

           nF1=(nF1-1);

       }

       sem_post((sem_t*)&f1_shared_area_ptr->mutex);



       if(send==1)
```

```
        write(pipe01[1],&received,sizeof(int));
        if(nF1==0)
        {
            printf("Thread1 finalizado: \n");
            //close(pipe01[1]);
            pthread_exit(0);
        }
    }
}

void* thread2 ( void *ptr )
{
    int received,rsc,i,send;
    while(1){
    send=0;
    sem_wait((sem_t*)&f1_shared_area_ptr->mutex);
        if ( nF1 > 0 ){
        received= f1_shared_area_ptr->int_value[0];
        send=1;
        printf("Thread2 recebeu %d: %d\n",nF1,received);
        for(i=0;i<nF1;i++){
            rsc=f1_shared_area_ptr->int_value[i+1];
            f1_shared_area_ptr->int_value[i] = rsc;
            }
        //f1_shared_area_ptr->int_value[nF1] =0;
```

```
        nF1=(nF1-1);



        }
    sem_post((sem_t*)&f1_shared_area_ptr->mutex);
    if(send==1)
    write(pipe02[1],&received,sizeof(int));



    if(nF1==0) {
    printf("Thread2 finalizado: \n");
    //close(pipe02[1]);
    pthread_exit(0);
    }



    }



}
```

singlethreads.c

```c
/**
* Este codigo cria os processos p1, p2 e p3. Apos sua criação
* é passado os valores inteiros para a fila f1 por meio da
* memoria compartilhada
**/
#include "ex03.h"


//Criando funcao para gerar numeros alatorios e compartilhar esses valores a f1
void singleProcess(){
   while(1)
   {
       sem_wait((sem_t*)&f1_shared_area_ptr->mutex);
       //Variavel n serve para compartilhar o numero de valores 1 a 10
       n = f1_shared_area_ptr->num;
        //Verificando se os valores nao passaram do limite da fila que e 10
       if ( n < 10 )
       {
           //Gerando valores de 1 a 1000
           f1_shared_area_ptr-> int_value[n]= 1 + (rand() % 999);
           //Incremento dos valores
           f1_shared_area_ptr-> num += 1;
           //Preparando compartilhamento na f1
```

```
                    printf("Foi produzido no campo [%d] o valor
[%d]\n",f1_shared_area_ptr->num, f1_shared_area_ptr->int_value[n]);

            //Verificando caso anterior ao 10 para enviar o kill

            if(n==9)

            {

                kill(f1_shared_area_ptr->PID, SIGUSR1);

                printf("Em aguardo: \n");

            }

        }

        //Compartilando recursos

        sem_post((sem_t*)&f1_shared_area_ptr->mutex);

    }

}



int main()

{

    int i;

    //Preparando processos

    pid_t pid1;

    pid_t pid2;



    //chave de identificação f1

    key_t key=1234;

    void *shared_memory = (void *)0;

    int shmid;
```

```
//Criação do segmento de memoria
shmid = shmget(key,MEM_F1,0666|IPC_CREAT);
//verificando situação
if ( shmid == -1 )
{
    printf("shmget falhou\n");
    exit(-1);
}
//Recebendo endereço de memoria
shared_memory = shmat(shmid,(void*)0,0);
//verificando situação
if (shared_memory == (void *) -1 )
{
    printf("shmat falhou\n");
    exit(-1);
}

f1_shared_area_ptr = (struct shared *) shared_memory;
if ( sem_init((sem_t *)&f1_shared_area_ptr->mutex,1,1) != 0 )
{
        printf("sem_init falhou\n");
        exit(-1);
}

f1_shared_area_ptr->num=0;
```

```
for(i=0;i<10;i++)
    f1_shared_area_ptr->int_value[i]=0;


//Criando filho 1
pid1=fork();
if( pid1 == 0 )
{
    singleProcess();
}


//Verificação da criação do PID
else if(pid1 > 0 )
{
    //Criando filho 2
    pid2=fork();
    if ( pid2 == 0 )
    {
        singleProcess();
    }
    else if (pid2 > 0 )
    {
        singleProcess();
    }
}
return 0;
```

```
}
```

final_part_p7.c

```
/**
* Este codigo cria os processos p5, p6 e p7. O processo 4 somente
* é acessado quando recebe um signal. Os recusos são compartilhados
*  pela fila f2.
**/

#include "ex03.h"

int main()
{
   //Preparacao para f2

   int i=0;
   //chave de identificação
   key_t key= 1243;
   void *shared_memory = (void *)0;
   int shmid;

      //Criando um novo segmento de memória compartilhada com size
equivalente a MEM_F2
   shmid = shmget(key,MEM_F2,0666|IPC_CREAT);
```

```
if ( shmid == -1 )
{
    printf("shmget falhou\n");
    exit(-1);
}


shared_memory = shmat(shmid,(void*)0,0);
if (shared_memory == (void *) -1 )
{
    printf("shmat falhou\n");
    exit(-1);
}

f2_shared_area_ptr = (struct shared *) shared_memory;

if ( sem_init((sem_t*)&f2_shared_area_ptr->mutex,1,1) != 0 )
{
    printf("sem_init falhou\n");
    exit(-1);
}

sem_wait((sem_t*)&f2_shared_area_ptr->mutex);
f2_shared_area_ptr->num=0;
for(i=0;i<10;i++) f2_shared_area_ptr->int_value[i]=0;
```

```
   sem_post((sem_t*)&f2_shared_area_ptr->mutex);

   moda = malloc(size * sizeof *moda);

   freq = malloc(size * sizeof *freq);

   printf("Processo P7 em aguardo: \n");

   pthread_t threads[3];


            for(i=0;i<3;i++)    pthread_create    (&threads[i],    NULL,
three_threads_p7, NULL);


   for(i=0;i<3;i++) pthread_join(threads[i], NULL);


   int maior=0,max_int_value=0,im,min_int_value;

   printf("\n O Processo p7 ja recebeu todos os 1000 valores : \n");

   for(i=0;i<size;i++)

   {

       if(freq[i]>=maior)

       {

           maior=freq[i];

           im=i;

       }

       if(moda[i]>=max_int_value)

       {

           max_int_value=moda[i];

       }
```

```
        if((i==0)||(min_int_value>=moda[i]))

        {

            min_int_value=moda[i];

        }

    }



    final_time = clock();

    /*

    printf("Tempo inicial: %ld\n",initial_time);

    printf("Tempo final: %ld\n",final_time);

    printf("clock: %ld\n",CLOCKS_PER_SEC);

    */

        total_time   =   (   (double)   (final_time   -   initial_time))   /
CLOCKS_PER_SEC;

     printf("\na)O tempo de execuçao total do programa foi de ------ :
%.3f segundos",total_time);

      printf("\nb)A quantidade de valores processados por process5,P6
foi-- : %d",counter_p5_p6);

    printf("\nc)O Valor que mais se repetiu é o ------------------- : %d
\n \\

    O maior valor computado foi ----------------- : %d \n \\

    O menor valor computado foi ----------------- : %d \n\n",

    moda[im],max_int_value,min_int_value);

    return 0;

}



void dates_interpreter(int in)
```

```
{
   int i,new=1;
   for(i=0;i<size;i++)
   {
       if(moda[i]==in)
       {
           freq[i]=freq[i]+1;
           new=0;
       }
   }
   if(new==1)
   {
       size++;
       moda = realloc(moda, size * sizeof(int));
       freq = realloc(freq, size * sizeof(int));
       freq[size-1]=1;
       moda[size-1]=in;
   }
}



void* three_threads_p7( void *ptr )
{
   int resorce,i,n7,P7;
   while(1)
```

```
    {
        sem_wait((sem_t*)&f2_shared_area_ptr->mutex);
        n7 = f2_shared_area_ptr->num;
        if ( n7 > 0 )
        {
            P7= f2_shared_area_ptr->int_value[0];
            for(i=0;i<n7;i++)
            {
                resorce=f2_shared_area_ptr->int_value[i+1];
                f2_shared_area_ptr->int_value[i] = resorce;
            }
            count++;
              printf("O processo p7 recebeu no campo [%d] o valor  [%d]
\n",count,P7);
            if(count > max_values)
            {
                return 0;
            }
            dates_interpreter(P7);
            f2_shared_area_ptr->num=(int)(n7-1);
        }

        if(count==max_values)
        {
```

```
            if(counter_p5_p6==0)
            {
                counter_p5_p6=f2_shared_area_ptr->counter_p5_p6;
                initial_time=f2_shared_area_ptr->t0;
            }
            sem_post((sem_t*)&f2_shared_area_ptr->mutex);
            pthread_exit(0);
        }

        else
        {
            sem_post((sem_t*)&f2_shared_area_ptr->mutex);
        }
    }
}
```